

COL100: Introduction to Computer Science

Lab 14: Program Efficiency and Sorting Algorithms

1 Introduction

1.1 Program Efficiency

Writing efficient programs is critical, as their performance often determines their utility, especially in real-time applications like aviation safety or bank database operations. Writing efficient code is challenging because optimal solutions often involve complex algorithms that trade simplicity for reduced computational complexity. To balance this trade-off effectively, understanding and estimating computational complexity is essential.

1.1.1 Time Complexity

Time complexity describes the growth of an algorithm's running time as a function of input size. It is expressed using Big-O notation, which provides an upper bound on the number of basic steps an algorithm performs.

Best Case

The best-case complexity refers to the minimum running time of an algorithm for the most favorable inputs. For example, in linear search, the best-case occurs when the target element is the first in the list:

```
1 def linear_search(L, x):  
2     for e in L:  
3         if e == x:  
4             return True  
5     return False
```

In this case, the algorithm executes in $O(1)$ time.

Worst Case

The worst-case complexity is the maximum running time for the least favorable inputs. For the same linear search example, this occurs when the target element is not in the list, requiring the algorithm to examine all elements: $O(n)$

Average Case

The average-case complexity represents the expected running time over all possible inputs, sometimes weighted by known distributions of input values.

1.1.2 Space Complexity

Space complexity measures the amount of memory an algorithm requires as a function of input size. It includes:

- Fixed part: Memory required for constants, program instructions, and variables.
- Variable part: Memory required for dynamic structures and recursion.

1.1.3 Asymptotic Notations

Asymptotic notation provides a formal way to describe the behavior of an algorithm as the size of its input approaches infinity. It helps identify the dominant factors in running time and provides an upper bound on growth rates.

- **Big O (O):** Upper bound on growth rate. Example: $f(x) \in O(x^2)$ means $f(x)$ grows no faster than x^2 asymptotically.
- **Big Theta (Θ):** Both upper and lower bound. Example: $f(x) \in \Theta(x^2)$ means $f(x)$ grows exactly as x^2 asymptotically.

1.1.4 Important Complexity Classes

Constant Complexity ($O(1)$)

The running time is independent of the size of the inputs. While most interesting programs are not in this class, individual operations like determining the length of a list or basic arithmetic fit here.

Logarithmic Complexity ($O(\log n)$)

Algorithms such as binary search have running times proportional to the logarithm of the input size. For example, converting an integer to a string representation (`int_to_str`) has complexity $\Theta(\log n)$.

Linear Complexity ($O(n)$)

Many algorithms that iterate over sequences, such as summing the digits of a string (`add_digits`), or recursive functions like `factorial`, are linear in complexity, i.e., $\Theta(n)$. Recursive functions may also have a space complexity proportional to their depth.

Log-Linear Complexity ($O(n \log n)$)

Algorithms like merge sort operate in $\Theta(n \log n)$ time, a combination of linear and logarithmic growth. These are practical for sorting and similar problems.

Polynomial Complexity ($O(n^k)$)

Quadratic algorithms, such as subset tests or intersection computations, have complexities like $\Theta(n_1 \cdot n_2)$. They grow faster than linear but are still manageable for reasonably sized inputs.

Exponential Complexity ($O(c^n)$)

Exponential algorithms, like generating power sets (`gen_powerset`), have complexities $\Theta(2^n)$. These algorithms quickly become impractical as input size grows. However, approximate or specialized solutions can make some exponential problems tractable.

1.2 Sorting algorithms

Sorting is a fundamental operation in computer science, involving the arrangement of data in a specified order (typically ascending or descending). Efficient sorting is crucial for optimizing the performance of other algorithms (such as searching and merging), and for organizing data for human readability and further processing.

1.2.1 Why Sorting Algorithms Matter

- **Efficiency:** Sorting large datasets efficiently is critical for performance-sensitive applications.
- **Enabling Other Algorithms:** Many algorithms, such as binary search, require sorted data.
- **Data Organization:** Sorted data is easier to analyze, visualize, and process.

1.2.2 Classification of Sorting Algorithms

Sorting algorithms can be broadly classified as:

- **Comparison-based sorts:** These use comparisons between elements, e.g., Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, Heap Sort.
- **Non-comparison-based sorts:** These use properties of the data, e.g., Counting Sort, Radix Sort, Bucket Sort.

1.2.3 Key Properties of Sorting Algorithms

- **Time Complexity:** Measures how the runtime grows with input size.
- **Space Complexity:** Measures extra memory used by the algorithm.
- **Stability:** A stable sort preserves the relative order of equal elements.

- **In-place:** An in-place sort uses only a constant amount of extra memory.

1.2.4 Common Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(n + k)$	Yes
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n + k)$	Yes
Shell Sort	$O(n \log n)$	$O(n^{1.5})$	$O(n^2)$	$O(1)$	No

Table 1: Comparison of common sorting algorithms

1.2.5 Brief Overview of Key Sorting Algorithms

Bubble Sort

Idea: Repeatedly step through the list, compare adjacent elements, and swap them if they are in the wrong order. The process repeats until the list is sorted.

Pseudocode:

```
for i = 0 to n-1
  for j = 0 to n-i-2
    if arr[j] > arr[j+1]
      swap arr[j], arr[j+1]
```

Properties: Simple but inefficient for large lists; stable and in-place.

Selection Sort

Idea: Repeatedly find the minimum element from the unsorted part and put it at the beginning.

Pseudocode:

```
for i = 0 to n-1
  min_idx = i
  for j = i+1 to n-1
    if arr[j] < arr[min_idx]
      min_idx = j
  swap arr[i], arr[min_idx]
```

Properties: In-place but not stable; always $O(n^2)$ comparisons.

Insertion Sort

Idea: Build the sorted array one element at a time by inserting each element into its correct position.

Pseudocode:

```
for i = 1 to n-1
    key = arr[i]
    j = i - 1
    while j >= 0 and arr[j] > key
        arr[j+1] = arr[j]
        j = j - 1
    arr[j+1] = key
```

Properties: Efficient for small or nearly sorted arrays; stable and in-place.

Merge Sort

Idea: Divide the array into halves, sort each half recursively, and merge the sorted halves.

Pseudocode:

```
function mergeSort(arr, l, r)
    if l < r
        m = (l + r) / 2
        mergeSort(arr, l, m)
        mergeSort(arr, m+1, r)
        merge(arr, l, m, r)
```

Properties: Always $O(n \log n)$ time; stable but not in-place.

Quick Sort

Idea: Select a pivot, partition the array around the pivot, and recursively sort the partitions.

Pseudocode:

```
function quickSort(arr, low, high)
    if low < high
        pi = partition(arr, low, high)
        quickSort(arr, low, pi-1)
        quickSort(arr, pi+1, high)
```

Properties: Fast in practice; in-place but not stable; worst-case $O(n^2)$.

1.2.6 Radix Sort

Idea: Sort numbers digit by digit, starting from the least significant digit, using a stable sort like counting sort at each step.

Pseudocode:

```
for d = least significant digit to most significant
    stableSort(arr, digit=d)
```

Properties: Linear time for fixed digit length; stable; not comparison-based.

1.2.7 Summary

Each sorting algorithm has its own strengths and weaknesses. The choice of algorithm depends on the size of the data, whether stability is required, and the characteristics of the input data (e.g., range, nearly sorted, etc.). In this lab, we will focus on implementing and understanding **Merge Sort**, but you are encouraged to explore and compare other algorithms as well.

2 Submission Problem

2.1 The Quest for the Kth Treasure

Long ago, in the kingdom of Algorithmia, a wise wizard stored magical treasures in an ancient array. To retrieve the k -th most powerful treasure, aspiring knights must solve the **Kth Smallest Element** problem. Your task is to write a program to help the knights find their prize!

Problem Statement

Given an array of n integers, find the k -th smallest element. Assume array elements are distinct for simplicity.

Input Format

- First line: Integer n (size of array)
- Second line: n space-separated integers
- Third line: Integer k

Output Format

Single integer: The k -th smallest element

Constraints

- $1 \leq k \leq n \leq 10^6$
- $-10^9 \leq \text{array elements} \leq 10^9$

Sample Quest

Input:

```
6
7 10 4 3 20 15
3
```

Output:

```
7
```

Explanation: The sorted array is [3, 4, 7, 10, 15, 20]. The 3rd element is 7.

3 Practice Problem

3.1 Optimizing Recursion

You are given a series which is defined recursively as follows:

$$\begin{aligned} T(n) &= 2 \cdot T(n-1) + 3 \cdot T(n-2), & \text{for } n \geq 3 \\ T(2) &= 1, \\ T(1) &= 1. \end{aligned}$$

You have to implement the following function which calculates the n^{th} element of this series.

```
1 def series(n: int) -> int:
2     pass
```

- Here $n \geq 1$
- First try a recursive implementation, and explore whether the program is fast enough for larger values of n .
- Can you write an $\mathcal{O}(n)$ algorithm without using recursion?
- Can you write an $\mathcal{O}(n)$ algorithm using recursion?
- Can you write an $\mathcal{O}(1)$ algorithm?

Sample Input

```
1 3
```

Additional Questions

- What is the Worst Case Time Complexity of the recursive implementation of this function?
- Why is recursion inefficient?
- What is the Worst Case Time Complexity of your algorithm?
- What is the best Worst Case Time Complexity of an algorithm which solves this problem? (Hint: Is an $\mathcal{O}(1)$ algorithm possible?)
- Can you argue for the **correctness** of each algorithm you write?

3.2 Generalized Fibonacci with Modulo

You are given a sequence defined recursively as follows:

$$\begin{aligned} G(n) &= (a \cdot G(n-1) + b \cdot G(n-2)) \mod m && \text{for } n \geq 3 \\ G(2) &= x \\ G(1) &= y \end{aligned}$$

You have to implement the following function which calculates the n^{th} element of this sequence for given values of a , b , m , x , and y .

```
1 def generalized_fibonacci(n: int, a: int, b: int, m: int, x: int, y:
   int) -> int:
2     pass
```

Constraints

- $n \geq 1$
- $0 \leq a, b, x, y, m \leq 10^9$
- $m \geq 1$ (the modulo operation ensures values stay manageable)

Tasks

1. Implement a recursive solution for this problem. Analyze the worst-case time complexity.
2. Optimize the recursive solution using memoization to improve efficiency.
3. Implement an iterative $\mathcal{O}(n)$ algorithm for the same.
4. Can you implement an $\mathcal{O}(\log n)$ algorithm using matrix exponentiation?
5. If possible, derive an $\mathcal{O}(1)$ solution (Hint: Can the problem be transformed into a closed-form expression for some special cases?).

3.2.1 Input

```
1 n = 5
2 a = 2
3 b = 3
4 m = 100
5 x = 1
6 y = 1
```

3.2.2 Output

```
1 89
```

3.3 Pair of Numbers in a List

3.3.1 Problem Statement

You are given an array of integers. Your task is to find the pair of numbers in the array whose sum is closest to zero. Implement a function that efficiently computes the result and analyzes its time complexity.

```
1 def closest_to_zero(arr: list[int]) -> tuple[int, int]:  
2     pass
```

- The function should return a tuple of two integers representing the pair with the closest sum to zero.
- Analyze the time complexity of your algorithm.
- Try to achieve an efficient algorithm with a time complexity better than $\mathcal{O}(n^2)$ if possible.

3.3.2 Test Case 1

Input

```
1 [-10, 20, -3, 5, 7, -8]
```

Output

```
1 (-3, 5)
```

Explanation The pair (-3, 5) has a sum of 2, which is the closest to zero among all possible pairs. The function should find this pair efficiently.

3.3.3 Test Case 2

Input

```
1 [1, 60, -10, 70, -80, 85]
```

Output

```
1 (-80, 85)
```

Explanation The pair (-80, 85) has a sum of 5, which is the closest to zero among all possible pairs.

3.3.4 Test Case 3

Input

```
1 [3, -2, 4, -1, -6, 8]
```

Output

```
1 (-2, 3)
```

Explanation The pair $(-2, 3)$ has a sum of 1, which is the closest to zero among all possible pairs.